

Cours de Python M1 Economie Finance - Partie 3

Université Paris Dauphine

L. Briens

Dauphine - CREST - Institut Polytechnique

Octobre 2024

Outline

1 Pandas

- Les bases de pandas
- Premiers DataFrames
- Les fonctions et méthodes de base

Pandas

Pandas est un package python pour la manipulation et l'analyse de données. Il organise les données dans des objets appelés *DataFrame*. Le package propose une multitude d'outils destinés à manipuler les données.

Pandas

Pandas est un package python pour la manipulation et l'analyse de données. Il organise les données dans des objets appelés *DataFrame*. Le package propose une multitude d'outils destinés à manipuler les données.

Installation de pandas :

```
# à écrire dans la console python
```

```
pip install pandas
```

```
# import du package (à mettre au début du script python)
```

```
import pandas as pd
```

Pandas

Pandas est un package python pour la manipulation et l'analyse de données. Il organise les données dans des objets appelés *DataFrame*. Le package propose une multitude d'outils destinés à manipuler les données.

Installation de pandas :

```
# à écrire dans la console python
pip install pandas

# import du package (à mettre au début du script python)
import pandas as pd
```

Pandas est un package utilisé en conjonction avec **numpy** et **matplotlib** qui permet de *visualiser*, *nettoyer*, *transformer* et *analyser* des bases de données.

Les Series et les DataFrames

Il existe 2 types d'objets principaux au sein du package pandas :

- **Les Series** : Structure de données unidimensionnelle hétérogène indexées en lignes (base 0).

Les Series et les DataFrames

Il existe 2 types d'objets principaux au sein du package pandas :

- **Les Series** : Structure de données unidimensionnelle hétérogène indexées en lignes (base 0).
- **Les DataFrame** : Structure de données multidimensionnelle hétérogène indexées en lignes et en colonnes (base 0).

Les Series et les DataFrames

Il existe 2 types d'objets principaux au sein du package pandas :

- **Les Series** : Structure de données unidimensionnelle hétérogène indexées en lignes (base 0).
- **Les DataFrame** : Structure de données multidimensionnelle hétérogène indexées en lignes et en colonnes (base 0).

Déclaration d'une pd.Séries pandas :

```
data = range(1,101)
pd.Series(data = data)
# ou
data = {'a': 1, 'b': 2, 'c': 3}
pd.Series(data = data)
# ou
from numpy.random import randint
data= randint(1,7,10000)
pd.Series(data=data)
```

A partir d'un dictionnaire

Création d'un premier DataFrame :

```
data = {'stocks': ['AMZN', 'TSLA', 'AAPL', 'Meta'],  
        'prices': [133.62, 889.36, 167.23, 161.11]}
```

```
df = pd.DataFrame(data)
```

```
df
```

Output :

	stocks	prices
0	AMZN	133.62
1	TSLA	889.36
2	AAPL	167.23
3	META	161.11

A partir d'un CSV, Excel ...

Import de données stockées sous d'autres formats :

```
df = pd.read_csv(my_csv.csv, sep = ',', header = True)
# argument possibles : use_cols, index_col, squeeze (to get a pd.series)
# prefix, na_values, true_values, false_values etc
df = pd.read_excel('my_csv.xlsx')
```

Output :

	stocks	prices
0	AMZN	133.62
1	TSLA	889.36
2	AAPL	167.23
3	META	161.11

A partir d'un lien HTML

Import de données directement depuis un lien HTML :

```
html = 'https://en.wikipedia.org/wiki/'  
html = html + 'List_of_largest_oil_and_gas_companies_by_revenue'  
stock_data = pd.read_html(html)  
pd.DataFrame(data = stock_data[0])
```

A partir d'un lien HTML

Import de données directement depuis un lien HTML :

```
html = 'https://en.wikipedia.org/wiki/'  
html = html + 'List_of_largest_oil_and_gas_companies_by_revenue'  
stock_data = pd.read_html(html)  
pd.DataFrame(data = stock_data[0])
```

Question

A quoi ressemble le dataframe extrait ?

Que pouvez-vous en dire ?

Qu'avons-nous à faire ?

Outline

1 Pandas

- Les bases de pandas
- Premiers DataFrames
- Les fonctions et méthodes de base

Ajout d'une colonne

Ajout de la colonne market cap

```
df = df.assign(market_cap=[1363, 2692, 931])  
# or  
df.insert(2, 'market_cap_bis', [1363, 2692, 931])  
  
df
```

Output :

	prices	market_cap	market_cap_bis
stocks			
AMZN	133.62	1363	1363
TSLA	889.36	2692	2692
AAPL	167.23	931	931

Ajout d'une colonne

Ajout de la colonne quantité d'actions via une fonction lambda :

```
df = df.assign(number_of_share=lambda x : x.market_cap*10**9/x.prices)
```

Output :

	prices	market_cap	market_cap_bis	number_of_share
stocks				
AMZN	133.62	1363	1363	1.020057e+10
TSLA	889.36	2692	2692	3.026896e+09
AAPL	167.23	931	931	5.567183e+09

Ajout d'une colonne

Ajout de la colonne market_cap via la méthode insert :

```
try :  
    df = df.insert(2, 'market_cap', [1363, 2692, 931])  
    print('Column insérée')  
except ValueError :  
    print('Colonne déjà présente')
```

Ajout d'une colonne

Ajout de la colonne `market_cap` via la méthode `insert` :

```
try :  
    df = df.insert(2, 'market_cap', [1363, 2692, 931])  
    print('Column insérée')  
except ValueError :  
    print('Colonne déjà présente')
```

Output :

Colonne déjà présente

Remarque

La gestion d'erreur permet, grâce à une structure `"try : ... except"`, d'anticiper si la colonne existe déjà et de ne pas déclencher une erreur. Il peut être assorti de plusieurs `except` et d'un `else`.

.loc[] et .iloc[]

Les propriétés *.loc* et *.iloc* permettent d'accéder à des lignes et colonnes spécifiques par le biais de leur nom (*loc*) ou de leur index (*iloc*).

Ci-dessous 2 façons de récupérer le prix et la market cap d'Amazon :

```
df.loc["AMZN",["prices","market_cap"]]  
df.iloc[0,[0,1]]
```

.loc[] et .iloc[]

Les propriétés `.loc` et `.iloc` permettent d'accéder à des lignes et colonnes spécifiques par le biais de leur nom (*loc*) ou de leur index (*iloc*).

Ci-dessous 2 façons de récupérer le prix et la market cap d'Amazon :

```
df.loc["AMZN", ["prices", "market_cap"]]  
df.iloc[0, [0, 1]]
```

Remarque

- *loc* et *iloc* s'utilisent avec des **crochets**.
- Pour passer plusieurs arguments il faut les passer sous forme de liste (cf exemple ci-dessus)

Accéder aux colonnes

L'accès aux colonnes se fait de 2 façons principales :

- `df.ColumnName`
- `df["ColumnName"]`

Les 2 commandes ci-dessous retournent la colonne prices du DataFrame :

```
df.prices  
df["prices"]
```

Accéder aux colonnes

L'accès aux colonnes se fait de 2 façons principales :

- `df.ColumnName`
- `df["ColumnName"]`

Les 2 commandes ci-dessous retournent la colonne `prices` du DataFrame :

```
df.prices  
df["prices"]
```

```
stocks  
AMZN    133.62  
TSLA    889.36  
AAPL    167.23  
Name: prices, dtype: float64
```

Liste des indices et des colonnes

Les attributs *index* et *columns* permettent de récupérer tous les indices et les colonnes du DataFrame.

```
df.index.tolist()
```

```
==> ?
```

Liste des indices et des colonnes

Les attributs *index* et *columns* permettent de récupérer tous les indices et les colonnes du DataFrame.

```
df.index.tolist()
==> ['?']
df.columns.tolist()
==> ['AMZN', 'TSLA', 'AAPL']
```

Liste des indices et des colonnes

Les attributs *index* et *columns* permettent de récupérer tous les indices et les colonnes du DataFrame.

```
df.index.tolist()
==> ['?']
df.index.tolist()
==> ['AMZN', 'TSLA', 'AAPL']
df.columns.tolist()
==> ['?']
df.columns.tolist()
==> ['prices', 'market_cap', 'market_cap_bis', 'number_of_share']
```

Liste des indices et des colonnes

Les attributs *index* et *columns* permettent de récupérer tous les indices et les colonnes du DataFrame.

```
df.index.tolist()
==> ['?']
df.columns.tolist()
==> ['?', 'prices', 'market_cap', 'market_cap_bis', 'number_of_share']
```

Remarque

Les attributs *index* et *columns* sont de type **"Index"**. Ce type d'objet à une méthode *.tolist()* qui permet de le convertir en liste.

NB : Le type **"Index"** est tout de même itérable. Il n'est donc pas nécessaire de passer en liste dès que l'on utilise *index* ou *columns*.

apply

La méthode `.apply()` permet d'appliquer une fonction (lambda ou pré-définie) à tout ou partie du DataFrame.

```
currency_col = ["prices", "market_cap", "market_cap_bis"]  
df[currency_col].apply(lambda x : x/1.0768)
```

apply

La méthode `.apply()` permet d'appliquer une fonction (lambda ou pré-définie) à tout ou partie du DataFrame.

```
currency_col = ["prices", "market_cap", "market_cap_bis"]  
df[currency_col].apply(lambda x : x/1.0768)
```

La commande ci-dessus applique une *lambda function* aux colonnes : prices, market_cap et market_cap_bis. **NB** : Ce n'est pas une modification du DataFrame, pour cela il faut réassigner les colonnes modifiées avec la fonction lambda.

	prices	market_cap	market_cap_bis
stocks			
AMZN	136.458333	1391.952614	1391.952614
TSLA	908.251634	2749.183007	2749.183007
AAPL	170.782271	950.776144	950.776144

head et tail

Les méthodes `.head()` et `.tail()` permettent de visualiser les premières et dernières lignes du DataFrame (respectivement).

```
df.head(2)
df.tail(2)
```

Output :

	prices	market_cap	market_cap_bis	number_of_share
stocks				
AMZN	133.62	1363	1363	1.020057e+10
TSLA	889.36	2692	2692	3.026896e+09

	prices	market_cap	market_cap_bis	number_of_share
stocks				
TSLA	889.36	2692	2692	3.026896e+09
AAPL	167.23	931	931	5.567183e+09

shape et dtypes

Les objets de type DataFrame et Series ont un attribut *shape* qui contient les dimensions de l'objet :

```
df.shape  
==> (3, 4)  
df.prices.shape  
==> (3,)
```

shape et dtypes

Les objets de type DataFrame et Series ont un attribut *shape* qui contient les dimensions de l'objet :

```
df.shape  
==> (3, 4)  
df.prices.shape  
==> (3,)
```

L'attribut *dtypes* contient les types de chacune des colonnes du DataFrame :

```
df.dtypes
```

Output :

```
prices          float64  
market_cap      int64  
market_cap_bis  int64  
number_of_share float64  
dtype: object
```

set_index

Ré-indexation du DataFrame avec les tickers des actions :

```
df.set_index('stocks', inplace = True)  
  
df
```

Output :

prices	
stocks	
AMZN	133.62
TSLA	889.36
AAPL	167.23
META	161.11

rename

La méthode `.rename()` permet de renommer les colonnes d'un DataFrame. Il prend en argument un dictionnaire où les **clés** sont les noms actuels et les **valeurs** sont les nouveaux noms de colonnes.

```
df.rename(columns = {"number_of_share":"shares"}, inplace = False)
```

rename

La méthode `.rename()` permet de renommer les colonnes d'un DataFrame. Il prend en argument un dictionnaire où les **clés** sont les noms actuels et les **valeurs** sont les nouveaux noms de colonnes.

```
df.rename(columns = {"number_of_share":"shares"}, inplace = False)
```

NB : *Inplace = False* (par défaut à *False*) sert à **ne pas modifier** l'objet "df", c'est donc une copie de df qui est retournée.

Output :

	prices	market_cap	market_cap_bis	shares
stocks				
AMZN	133.62	1363	1363	1.020057e+10
TSLA	889.36	2692	2692	3.026896e+09
AAPL	167.23	931	931	5.567183e+09

drop

La méthode `.drop()` permet de supprimer des lignes et/ou des colonnes. Il possède aussi l'argument `inplace` (booléen) qui précise si on modifie l'objet ou si on crée une copie. Il faut préciser l'axe sur lequel on intervient, `axis = 1` ou `0` (colonnes ou lignes).

```
df.drop(["market_cap", "number_of_share"], axis = 1)
```

Output :

	prices	market_cap_bis
stocks		
AMZN	133.62	1363
TSLA	889.36	2692
AAPL	167.23	931

info

La méthode `.info()` permet d'obtenir les informations de base sur le DataFrame telles que les types de variables, la mémoire utilisée, le nombre de données par colonnes et leurs noms.

```
df.info()
```

Output :

```
<class 'pandas.core.frame.DataFrame'>
Index: 3 entries, AMZN to AAPL
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype
---  -
0   prices          3 non-null     float64
1   market_cap      3 non-null     int64
2   market_cap_bis  3 non-null     int64
3   number_of_share 3 non-null     float64
dtypes: float64(2), int64(2)
memory usage: 120.0+ bytes
```

describe

La méthode `.describe()` permet d'obtenir les statistiques descriptives basiques d'un DataFrame. Cela permet de s'approprier la base de données en amont du data engineering.

```
df.describe()
```

Output :

	prices	market_cap	market_cap_bis	number_of_share
count	3.000000	3.000000	3.000000	3.000000e+00
mean	396.736667	1662.000000	1662.000000	6.264882e+09
std	426.955173	917.785923	917.785923	3.637373e+09
min	133.620000	931.000000	931.000000	3.026896e+09
25%	150.425000	1147.000000	1147.000000	4.297039e+09
50%	167.230000	1363.000000	1363.000000	5.567183e+09
75%	528.295000	2027.500000	2027.500000	7.883876e+09
max	889.360000	2692.000000	2692.000000	1.020057e+10

Méthodes statistiques usuelles

Les méthodes de statistiques usuelles sont toutes implémentées et accessible facilement. Ci-dessous une liste non-exhaustive :

- *.mean()*
- *.median()*
- *.std()*
- *.var()*
- *.corr()*
- *.cov()*
- *.min()*
- *.max()*
- *.count()*
- *.pct_change()*
- ...

Exercice

Remettre en forme le dataset récupéré : "*List of largest oil and gas companies by revenue*" scrappé sur Wikipédia juste avant.

Consigne : En faire un dataframe de scalaires sur lequel on peut calculer toutes les statistiques descriptives usuelles.

Exercice

Remettre en forme le dataset récupéré : *"List of largest oil and gas companies by revenue"* scrappé sur Wikipédia juste avant.

Consigne : En faire un dataframe de scalaires sur lequel on peut calculer toutes les statistiques descriptives usuelles.

Aide

- Utiliser `.apply()` (méthode de DataFrame et Series)
- Utiliser `.split()` (méthode de str)
- Utiliser `.startswith()` (méthode de str)
- Utiliser `.astype()` (méthode de DataFrame et Series)

Application : DataFrame CAC40

Télécharger le nouveau DataSet via le lien suivant :



Ouvrir le jeu de données et conserver les données entre le 30 décembre 2011 et le 30 décembre 2016 :

```
df_CAC = pd.read_excel("data_CAC.xlsx",  
                        sheet_name="Prices",  
                        index_col="Dates").loc["2011-12-30":"2016-12-30",:]
```

Télécharger le notebook "TP CAPM CAC 40" et commencer le TP.